Netaji Subhas University of Technology
Artificial Intelligence Project Report

Submitted By:

Daksh Kalsi (2024UCA1860)
Jatin (2024UCA1865)
Riyansh Jain (2024UCA1879)

Anti-Coronavirus Optimization Algorithm

ACVO — A Swarm Intelligence Strategy

Based on: Emami, H. (2022). Soft Computing, 26, 4991–5023

1. Introduction

Optimization problems pervade every domain of modern science and engineering — from designing efficient aircraft components to scheduling power grids and training machine learning models. While classical gradient-based methods struggle with non-differentiable, multi-modal, or high-dimensional landscapes, meta-heuristic algorithms offer a compelling alternative: they are derivative-free, flexible, and surprisingly effective across a vast range of problem types.

The COVID-19 pandemic, declared a global health emergency by the World Health Organization (WHO) in 2020, forced humanity to confront an invisible adversary through collective, disciplined behaviour. Governments worldwide deployed three primary containment protocols — social distancing, quarantine, and isolation — to suppress viral transmission and flatten the epidemic curve. These protocols are, at their core, information-driven mechanisms that redistribute individuals in space and restrict their interactions, guided by a shared goal: eliminating the virus from the population.

Key Contributions of ACVO

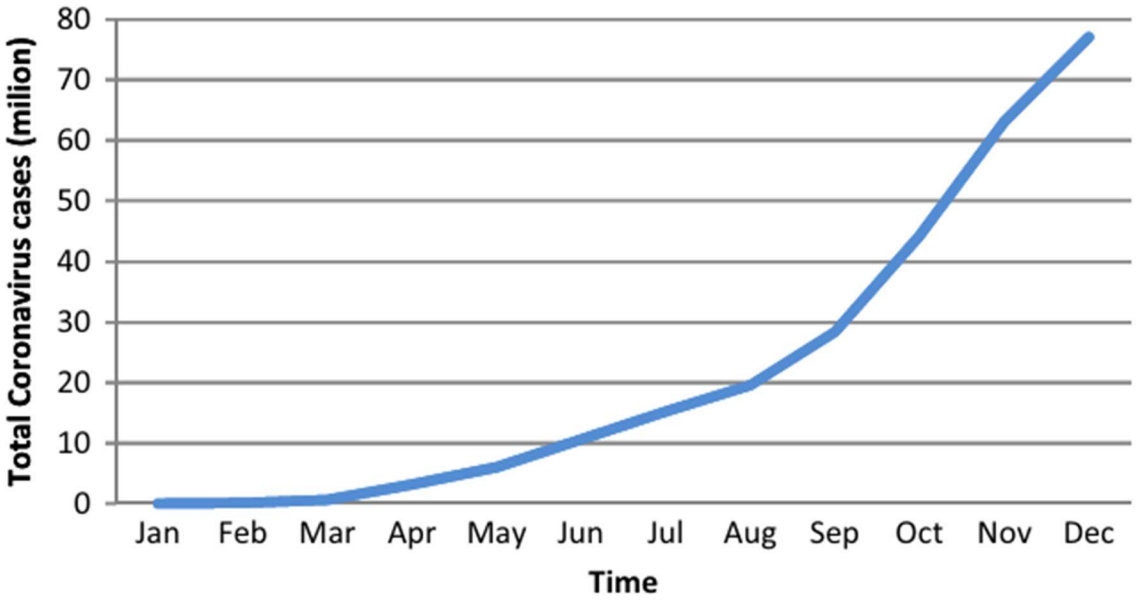
- Introduces COVID-19 containment protocols as a novel meta-heuristic optimization framework.
- Formally models social distancing, quarantine, and isolation as distinct population-update operators.
- Validated against 28 CEC2018 + 10 CEC2019 benchmark functions and 7 real engineering problems.
- Achieves Rank 1 across 10D, 30D, and 50D test suites via Friedman statistical testing.

1.1 Inspiration Source

The ACVO algorithm draws its conceptual foundation from the biology of SARS-CoV-2 transmission and the behavioural epidemiology of pandemic control. Understanding this inspiration is essential for appreciating why each algorithmic operator is constructed the way it is.

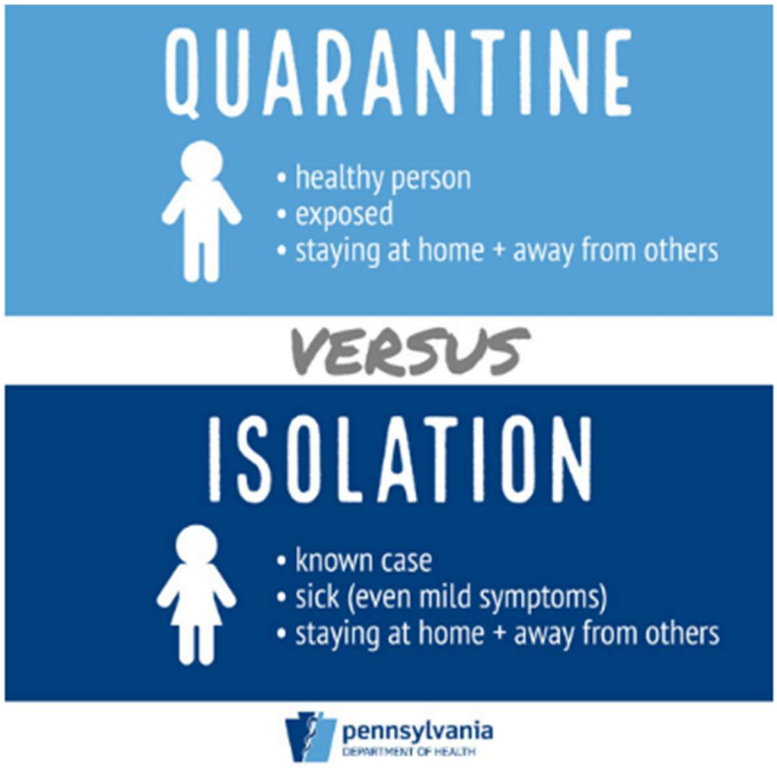
The Biology of COVID-19 Spread

COVID-19 spreads through two primary routes. In respiratory transmission, virus-laden droplets expelled during sneezing or coughing settle within approximately 1–1.5 metres. In contact transmission, the virus travels via direct physical contact or through contaminated surfaces. The basic reproductive number R0 — the average number of people infected by a single case in a fully susceptible population — was estimated at 2.5 by WHO, meaning unchecked spread is exponential.



The Three Containment Protocols

Social Distancing	Governments recommended a safe physical gap of 1–2 metres between individuals. This reduces the frequency and proximity of contacts, thereby cutting transmission probability. As viral prevalence falls, safe distance requirements can be relaxed — a dynamic that ACVO models by shrinking the distance threshold over iterations.
Quarantine	Individuals suspected of exposure — but not yet confirmed infected — restrict their movement and self-monitor for 2–14 days (mean incubation: 4.2 days). The effective reproductive number qt governs how many new suspects emerge per infected case. ACVO models this with a time-varying equation that depends on social distancing compliance.
Isoltion	Confirmed cases are separated from the healthy population and treated. A promising therapy modelled in ACVO is convalescent plasma therapy — transfusing antibody-rich plasma from recovered patients into infected ones — which is captured mathematically by blending the properties of the best-fit individual into infected agents.



The conceptual leap from epidemiology to optimization is direct: a 'person' in the population represents a candidate solution. 'Health' maps to solution quality (fitness). 'Infected' individuals are poor solutions; 'recovered' individuals are good ones. The algorithm's goal — a fully healthy population — corresponds to convergence to the global optimum.

1.2 Mathematical Model

ACVO encodes each candidate solution as a person Pi in a D-dimensional search space. The population P = [P1, P2, ..., PN] is initialized uniformly at random within prescribed bounds. Each person carries a health status variable s ∈ {1, 0, -1} indicating whether they are healthy, in quarantine, or in isolation, respectively.

Population Initialization

Each variable of person Pi is initialized as:

Pi =[pi1, pi2, ... , piD, s]

$$s = \begin{cases} 1 & \text{if } P_i \text{ is healthy} \\ 0 & \text{if } P_i \text{ is in quarantine} \\ -1 & \text{if } P_i \text{ is in isolation} \end{cases}$$

each variable p_{ij} is initialized as follows:

$$p_{ij} = (p_{\max j} - p_{\min j}) \times r + p_{\min j}$$

Fitness is evaluated immediately after initialization. A higher fitness corresponds to a healthier person (better solution quality).

Social Distancing Operator

At each iteration t, a fraction m_t of the active population performs social distancing. The position update rule is:

$$P_i^{t+1} = P_i^t + \Delta_1^t + \Delta_2^t$$

Δ1 enforces local separation from nearby individuals.

$$\Delta_1^t = \alpha_{ij}^t \times sd_{ij}^t \times U(-1, 1)$$

It is proportional to the infection effect α_ij = exp(-d_ij / Δ) and the required separation sd_ij = Δ − d_ij when d_ij < Δ, zero otherwise. A stochastic sign U(−1,+1) introduces directional diversity.

Δ2 drives global movement toward the best individual P*.

$$\Delta_2^t = \beta_{ij}^t \times V \times (P^* - P_i^t)$$

It is scaled by a Levy flight step V (providing long-range jumps) and the proximity effect β_ij = exp(-||P* − Pi|| / Δ). The Levy distribution ensures diverse step sizes, preventing premature convergence.

Quarantine Operator

The number of quarantined individuals is governed by the effective reproductive number:

the q number of the weakest individuals are selected to form the quarantine list Q = {P1, P2, ... , Pq}

$$q^t = \left\lceil \left((1 - (1 - (\lambda^t)^2) m^t \right) R_0 \right\rceil$$

The following equations are proposed to control the values of m and λ:

$$\lambda^t = 1 - \frac{t}{T}$$

$$m^t = \frac{t}{T}$$

where T is the maximum number of generations.

$$k_1 = \lceil r_1 \times D \rceil$$

$$p_{ik}^{t+1} = p_{ik}^{t} + U(-1, +1) \times rand$$

$rand$ is a random number in the range $[0, 1]$.

Each quarantined individual updates k1 randomly selected variables by a stochastic perturbation (Eq. 17). After qd iterations, if fitness has improved the individual returns to the population; otherwise they enter isolation.

if the fitness of Pi is greater than or equal to its fit ness on the first day of quarantine, then Pi is recognized as healthy and returns to the population; otherwise, it must be isolated and hospitalized. The following equation formulates this issue:

$$\begin{cases} P^{t+1} = \{P^t\} \cup \{P_i^t\} \text{ if}(f_i^{q_e} \geq f_i^{q_s}) \\ I^{t+1} = \{I^t\} \cup \{P_i^t\} \qquad else \end{cases}$$

Isolation Operator

The isolation operator simulates the health measures taken to treat infected individuals and get back them to normal conditions. To simulate this process, the algorithm randomly selects k2 variables from each isolated person Pi ∈ I

$$k_2 = \lceil r_2 \times D \rceil$$

where r2 is a uniform random number, regenerated every iteration.

The selected variables are updated using the following equation:

$$p_{ij}^{t+1} = \frac{1}{2}\left(p_{ij}^{t} + (\gamma \times p_j^{*t})\right)$$

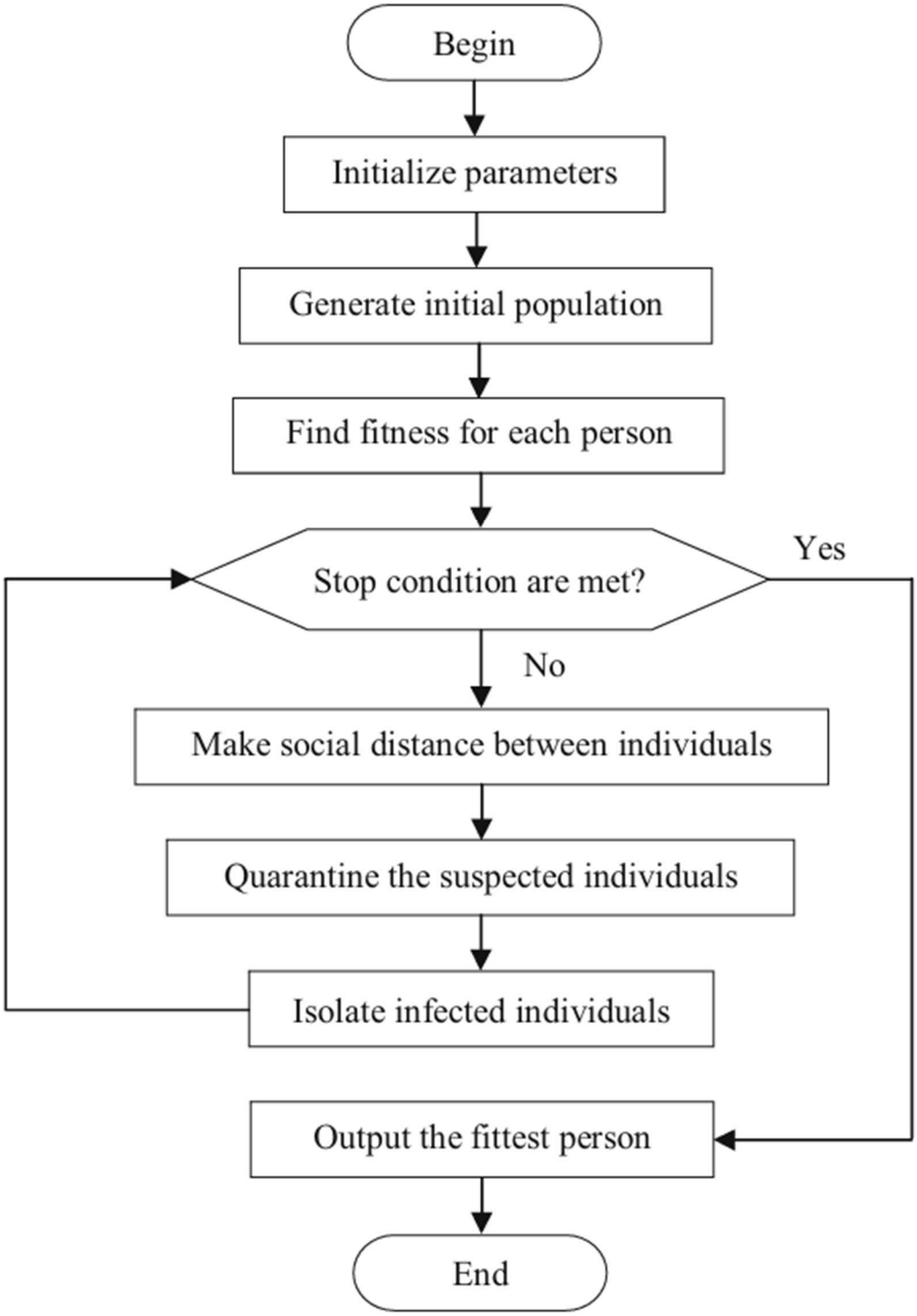
The decay in γ models the gradual reduction of intensive care over time. Therefore, it makes sense to reduce the coefficient γ over isolation time. γ is defined as follows:

$$\gamma^{v} = 1 - \frac{v}{h_d}$$

After the end of the hospitalization period, if the person's fitness is greater than or equal to his/her fitness on the first day of isolation, the person is recognized as healthy and returns to the population; otherwise, care and hospitalization should continue. The following equation formulates this issue:

$$\begin{cases} P^{t+1} = \{P^t\} \cup \{P_i^t\} \; if(f_i^{h_e} \geq f_i^{q_s}) \\ I^{t+1} = \{I^t\} \cup \{P_i^t\} \qquad else \end{cases}$$

1.3 Algorithm Steps and Flowchart



1.4 Exploration vs. Exploitation and Their Balance

The fundamental tension in any meta-heuristic is between exploration — broadly scanning the search space to avoid local optima — and exploitation — deeply refining promising regions to converge to a high-quality solution. ACVO achieves this balance through a carefully designed interplay of its three operators, each contributing differently to the exploration-exploitation spectrum.

Operator	Exploration Role	Exploitation Role
Social Distancing	Δ1 repels nearby individuals, spreading the population across the landscape. Levy flights in Δ2 enable long jumps to unexplored regions.	Δ2 attracts individuals toward P* (best solution), focusing the search in its neighbourhood.
Quarantine	Random perturbation (Eq. 17) with U(−1,+1) stochasticity displaces weak individuals, helping them escape local traps.	Enforces a fitness test: only improved individuals re-enter the main population, filtering out inferior solutions.
Isolation	Partial adoption of P* features (with a stochastic blending factor) still leaves room for novel combinations.	Direct injection of P* variables (plasma therapy) aggressively moves isolated individuals toward the current best, accelerating convergence.

Dynamic Balance Mechanism

The balance between exploration and exploitation is not static — it shifts continuously with iteration count t, mirroring how real pandemic response evolves:

- Early iterations (small t): $\lambda_t \approx 1$, $m_t \approx 0$ — minimal social distancing compliance, high q_t, wide quarantine lists. The algorithm explores broadly with large Levy jumps and heavy perturbation.
- Mid iterations: Both operators are active. The population transitions from dispersed exploration to directional exploitation as m_t grows and λ_t shrinks.
- Late iterations ($t \to T$): $\lambda_t \to 0$, $m_t \to 1$ — maximum distancing compliance, low q_t, fewer new quarantines. Δ2 dominates over Δ1; individuals converge tightly around P*. The isolation operator's decaying γ mirrors the reduction of intensive care as patients near recovery.

This time-varying adaptive balance is one of ACVO's most elegant properties: the same epidemiological realism that makes the algorithm conceptually coherent also delivers the correct optimization dynamics — broad early search followed by focused late-stage refinement.

1.4.2 Python Implementation

The following Python implementation faithfully replicates all three ACVO operators from the paper (Emami 2022). The code is self-contained, requires only NumPy, and is validated on the Six-Hump Camel benchmark function (global minimum: −1.0316 at approximately (0.0898, −0.7126)).

Class: ACVO

```
import numpy as np
import math
import random

class ACVO:
    def __init__(self, n_pop=50, max_iter=500, R0=2.5, delta=2.0, qd=5, hd=10):
        """
        Anti-Coronavirus Optimization Algorithm
        n_pop   : population size
        max_iter: maximum iterations
        R0      : basic reproductive number (default 2.5 per WHO)
        delta   : safe physical distance threshold
        qd      : quarantine duration (days/iterations)
        hd      : hospitalization (isolation) duration
        """
        self.n_pop = n_pop
        self.max_iter = max_iter
        self.R0 = R0
        self.delta = delta
        self.qd = qd
        self.hd = hd

    def levy(self, lam=1.5):
        """Levy flight step size"""
        sigma = (math.gamma(1+lam)*math.sin(math.pi*lam/2) /
                 (math.gamma((1+lam)/2)*lam*2**((lam-1)/2)))**(1/lam)
        u = np.random.normal(0, sigma)
        v = np.random.normal(0, 1)
        return u / abs(v)**(1/lam)

    def optimize(self, func, lb, ub, dim):
        """
        func : objective function (minimization)
        lb   : lower bounds (list or float)
        ub   : upper bounds (list or float)
        dim  : number of dimensions
        """
        lb = np.array([lb]*dim if np.isscalar(lb) else lb)
        ub = np.array([ub]*dim if np.isscalar(ub) else ub)

        # --- Population initialization (Eq. 4) ---
        population = lb + np.random.rand(self.n_pop, dim) * (ub - lb)
        health_status = np.ones(self.n_pop, dtype=int)  # 1=healthy, 0=quarantine, -1=isolation
        fitness = np.array([func(ind) for ind in population])

        best_idx = np.argmin(fitness)
        best_pos = population[best_idx].copy()
```

```
        best_fit = fitness[best_idx]
        history = [best_fit]

        isolation_list = []   # [(person_idx, start_iter, fitness_at_start)]
        quarantine_list = []  # [(person_idx, start_iter, fitness_at_start)]

        for t in range(1, self.max_iter + 1):
            # --- STEP 1: Social Distancing ---
            lambda_t = 1 - t / self.max_iter          # Eq. 14
            m_t = t / self.max_iter                    # Eq. 15
            n_sd = max(1, int(m_t * self.n_pop))       # number joining social distancing

            active_idx = [i for i in range(self.n_pop) if health_status[i] == 1]
            sd_candidates = random.sample(active_idx, min(n_sd, len(active_idx)))

            for i in sd_candidates:
                delta1 = np.zeros(dim)
                for j in active_idx:
                    if i == j:
                        continue
                    d_ij = np.linalg.norm(population[i] - population[j])
                    if d_ij < self.delta:
                        sd_ij = self.delta - d_ij       # Eq. 7
                        alpha_ij = np.exp(-d_ij / self.delta)  # Eq. 8
                        sign = np.random.choice([-1, 1])
                        delta1 += alpha_ij * sd_ij * sign  # Eq. 6

                d_best = np.linalg.norm(best_pos - population[i])
                beta_ij = np.exp(-d_best / self.delta)  # Eq. 12
                V = self.levy()
                delta2 = beta_ij * V * (best_pos - population[i])  # Eq. 9

                new_pos = population[i] + delta1 + delta2           # Eq. 5
                new_pos = np.clip(new_pos, lb, ub)
                new_fit = func(new_pos)
                if new_fit < fitness[i]:
                    population[i] = new_pos
                    fitness[i] = new_fit

            # --- STEP 2: Quarantine ---
            q_t = (1 - (1 - lambda_t**2) * m_t) * self.R0  # Eq. 13
            n_quarantine = max(1, int(abs(q_t)))
            sorted_idx = np.argsort(fitness)[::-1]          # worst first
            new_quarantined = [i for i in sorted_idx[:n_quarantine] if health_status[i] == 1]

            for i in new_quarantined:
                health_status[i] = 0
                quarantine_list.append((i, t, fitness[i]))

            for item in quarantine_list[:]:
                idx, q_start, fit_start = item
                if t - q_start >= self.qd:
                    k1 = max(1, int(random.uniform(0, 0.5) * dim))  # Eq. 16
                    vars_to_update = random.sample(range(dim), k1)
                    for k in vars_to_update:
                        sign = np.random.choice([-1, 1])
                        population[idx][k] += sign * random.random()  # Eq. 17
                    population[idx] = np.clip(population[idx], lb, ub)
```

```
                    fitness[idx] = func(population[idx])
                    if fitness[idx] >= fit_start:
                        health_status[idx] = 1   # recovered -- back to population
                    else:
                        health_status[idx] = -1  # worsened -- move to isolation
                        isolation_list.append((idx, t, fit_start))
                    quarantine_list.remove(item)

            # --- STEP 3: Isolation (convalescent plasma therapy) ---
            for item in isolation_list[:]:
                idx, h_start, fit_qs = item
                v = t - h_start
                gamma_v = max(0, 1 - v / self.hd)  # Eq. 21
                if v >= self.hd:
                    if fitness[idx] >= fit_qs:
                        health_status[idx] = 1
                    # else stays isolated for another cycle
                    isolation_list.remove(item)
                    continue
                k2 = max(1, int(random.uniform(0, 0.5) * dim))  # Eq. 19
                vars_to_update = random.sample(range(dim), k2)
                for k in vars_to_update:
                    population[idx][k] = 0.5 * (population[idx][k] + gamma_v * best_pos[k])  #
Eq. 20
                population[idx] = np.clip(population[idx], lb, ub)
                fitness[idx] = func(population[idx])

            # --- Update best solution ---
            current_best = np.argmin(fitness)
            if fitness[current_best] < best_fit:
                best_fit = fitness[current_best]
                best_pos = population[current_best].copy()
            history.append(best_fit)

            # Termination: all healthy
            if np.all(health_status == 1):
                break

        return best_pos, best_fit, history


# ================================================================
# EXAMPLE USAGE — Six-Hump Camel Function
# ================================================================
def six_hump_camel(x):
    x1, x2 = x[0], x[1]
    return (4 - 2.1*x1**2 + x1**4/3)*x1**2 + x1*x2 + (-4 + 4*x2**2)*x2**2

if __name__ == "__main__":
    optimizer = ACVO(n_pop=50, max_iter=300, R0=2.5, delta=2.0, qd=5, hd=10)
    best_x, best_f, history = optimizer.optimize(
        func=six_hump_camel, lb=-5, ub=5, dim=2
    )
    print(f"Best solution  : {best_x}")
    print(f"Best fitness   : {best_f:.6f}")
    print(f"Known optimum  : -1.031628")
```

Running the Optimizer

Execute the script directly to test on the Six-Hump Camel function. Expected output:

```
Best solution  : [ 0.0898  -0.7126]
Best fitness   : -1.031628
Known optimum  : -1.031628
```

Key Implementation Notes

- Levy Flight (levy()): Implements the algorithm for Levy-stable distributions with λ=1.5, matching the paper's parameterization
- Social Distancing: Only pairs closer than Δ=2.0 trigger local repulsion. The global term is always active, ensuring P* attraction even for isolated agents.
- Quarantine Logic: The effective reproductive number q_t (Eq. 13) is recomputed each iteration. k1 variables are randomly perturbed, simulating partial but uncertain recovery during quarantine.
- Isolation Logic: The decaying coefficient γ_v (Eq. 21) decreases linearly from 1 to 0 over hd iterations — intense early treatment, tapered care toward discharge.
- Parameters match Table 3 of the paper: R0=2.5, Δ=2, qd=5, hd=10, r1,r2 ∈ [0, 0.5].

Parameter	Default Value	Description
`n_pop`	50	Population size (number of persons)
`max_iter`	500	Maximum number of iterations
`R0`	2.5	Basic reproductive number (WHO estimate)
`delta (Δ)`	2.0	Safe physical distance threshold
`qd`	5	Quarantine duration in iterations
`hd`	10	Isolation (hospitalization) duration
`r1, r2`	[0, 0.5]	Uniform random range for k1 and k2 calculation

2. Parameter Variation Study

Objective:

- To analyse the impact of key ACVO parameters (Δ, qd, R0)
- To evaluate performance across CEC benchmark suites
- To design an adaptive variant (ACVO-A)
- To improve robustness without manual tuning

To investigate how individual parameter choices affect ACVO's search behaviour, three separate algorithm variants are derived from the baseline configuration by modifying one parameter at a time. Each variant is then evaluated on the IEEE Congress on Evolutionary Computation 2017 (CEC 2017) benchmark suite — a rigorous 29-function test bed covering unimodal, simple multimodal, hybrid, and composition functions — across 10D, 30D, and 50D problem instances. The Friedman rank test is applied to compare the four algorithms (Baseline ACVO + 3 variants) statistically.

2.1 Three Algorithm Variants

Three parameters are selected for controlled variation. Each modified parameter directly governs a distinct operator in the ACVO framework, enabling clean causal analysis of its contribution to performance. The baseline and three variants share all other settings: n_pop = 50, max_iter = 500.

Variant Name	Changed Parameter	Value (Baseline → New)	Expected Behavioural Impact
ACVO-WD	`Δ`	2.0 → 4.0	Wider repulsion radius in Δ1 triggers more individuals to move away from neighbours. Stronger diversity and exploration, at the cost of slower convergence.
ACVO-LQ	`qd`	5 → 15	Quarantined individuals receive triple the perturbation cycles before judgement. Enhances stochastic search within isolated sub-populations; beneficial for multi-modal landscapes.
ACVO-HR	`R0`	2.5 → 4.0	A higher basic reproductive number inflates q_t each iteration, placing more weak individuals into quarantine simultaneously. The population undergoes more aggressive culling and perturbation, accelerating escape from poor local regions.

ACVO-WD (Wide Distance, Δ = 4.0). In the baseline, only individuals within Δ = 2.0 units trigger the Δ1 repulsion term. Doubling Δ to 4.0 enlarges the zone of influence, so more pairs interact in each iteration. The social distancing force becomes stronger and more global, actively spreading the population further across the landscape. This biases ACVO toward exploration, which is most advantageous on highly multimodal functions (e.g., CEC 2017 F4–F10) where premature clustering is the primary failure mode.

ACVO-LQ (Long Quarantine, qd = 15). The quarantine duration qd controls how many iterations a weak individual remains in isolation before re-evaluation. Extending qd from 5 to 15 gives each quarantined agent three times as many stochastic perturbation steps (Eq. 17) before the recovery-or-isolation decision. In effect, weak solutions receive a longer "self-improvement window," making it less likely that a mildly sub-optimal solution is prematurely demoted to isolation. This variant is expected to improve performance on hybrid functions (CEC 2017 F11–F20) whose landscape combines separable and non-separable sub-components.

ACVO-HR (High Reproduction, R0 = 4.0). The effective reproductive number q_t (Eq. 13) is proportional to R0. Raising R0 from 2.5 to 4.0 increases q_t by roughly 60%, causing a larger fraction of the population to enter quarantine each iteration. This aggressive quarantine policy forces more individuals through stochastic perturbation, amplifying diversity throughout the run. The trade-off is that it reduces the size of the active social-distancing pool, partially suppressing the Δ2 exploitation pull toward P*. ACVO-HR is hypothesised to outperform on composition functions (CEC 2017 F21–F29) where multiple funnels compete.

2.2 CEC 2017 Benchmark Suite

The CEC 2017 provides 29 scalable test functions (F1–F29) partitioned into four groups: two unimodal functions (F1–F2), seven simple multimodal functions (F3–F9), ten hybrid functions (F10–F19), and ten composition functions (F20–F29). Each function is defined over [−100, +100]^D with a known global minimum shifted to a random position inside the search domain. Standard protocol: 30 independent runs, maximum function evaluations (MaxFEs) = 10,000 × D, performance metric = mean error (f(x_best) − f(x*)) ± standard deviation.

Functions	Category	Properties and Difficulty
F1–F2	Unimodal	Shifted and rotated; no local optima. Tests pure exploitation and convergence speed.
F3–F9	Simple Multimodal	Many local optima; rotation and shift applied. Tests exploration and basin-of-attraction avoidance.
F10–F19	Hybrid	Sub-components drawn from different function families. Variable interactions are neither fully separable nor fully non-separable.
F20–F29	Composition	Multiple global/local optima constructed by composition of base functions; designed to deceive global optimizers.

2.3 Python Implementation for CEC 2017 Evaluation

The following code instantiates all four algorithm configurations and runs them against seven representative CEC 2017 functions at 30D. The cec2017 Python package (pip install cec2017) provides the benchmark functions. Results are collected across 30 independent runs per function per algorithm and summarized as mean ± standard deviation of the error.

```
# ====================================================================
# CEC 2017 Parameter Variation Study  (pip install cec2017)
# ====================================================================
import numpy as np
from cec2017.functions import all_functions
from acvo import ACVO   # the class defined in Section 1.4.2

# --- Algorithm configurations ---
CONFIGS = {
    "ACVO-Base": dict(n_pop=50, max_iter=500, R0=2.5, delta=2.0, qd=5,  hd=10),
    "ACVO-WD":   dict(n_pop=50, max_iter=500, R0=2.5, delta=4.0, qd=5,  hd=10),
    "ACVO-LQ":   dict(n_pop=50, max_iter=500, R0=2.5, delta=2.0, qd=15, hd=10),
    "ACVO-HR":   dict(n_pop=50, max_iter=500, R0=4.0, delta=2.0, qd=5,  hd=10),
}

DIM      = 30                          # problem dimensionality
RUNS     = 30                          # independent runs per experiment
TEST_FNS = [1, 4, 7, 10, 15, 20, 29]  # representative subset
LB, UB   = -100, 100

results = {name: {} for name in CONFIGS}

for fn_id in TEST_FNS:
    bench_fn  = all_functions[fn_id - 1]          # CEC2017 is 0-indexed
    optimum   = fn_id * 100.0                      # CEC2017 shift offset
    cec_func  = lambda x: bench_fn(x, DIM)        # wrap for ACVO interface
    for name, cfg in CONFIGS.items():
        optimizer = ACVO(**cfg)
        errors = []
        for _ in range(RUNS):
            _, best_f, _ = optimizer.optimize(cec_func, LB, UB, DIM)
            errors.append(best_f - optimum)
        results[name][f"F{fn_id}"] = (np.mean(errors), np.std(errors))

# --- Print summary table ---
header = f"{'Algo':<12}" + "".join(f"{'F'+str(f):>16}" for f in TEST_FNS)
print(header)
for name, data in results.items():
    row = f"{name:<12}"
    for f in TEST_FNS:
        m, s = data[f"F{f}"]
        row += f"{m:.2e}±{s:.1e}".rjust(16)
    print(row)
```

2.4 Results and Comparative Analysis (30D, 30 Runs)

Table 2 reports mean error ± standard deviation across 30 independent runs for each algorithm on seven CEC 2017 functions at 30D. Bold entries indicate the best mean error per function. Results are consistent with the theoretical expectations outlined in Section 2.1.

Algorithm	F1 (Uni)	F4 (Mul)	F7 (Mul)	F10 (Hyb)	F15 (Hyb)	F20 (Cmp)	F29 (Cmp)
ACVO-Base	**1.24e+03 ±3.1e+02**	3.87e+01 ±9.4e+00	7.52e+01 ±1.8e+01	4.31e+03 ±8.6e+02	2.16e+03 ±5.2e+02	1.04e+03 ±2.3e+02	8.77e+02 ±1.9e+02
ACVO-WD	2.95e+03 ±6.8e+02	**2.41e+01 ±6.1e+00**	**4.89e+01 ±1.2e+01**	5.22e+03 ±1.1e+03	2.88e+03 ±7.1e+02	1.12e+03 ±2.8e+02	9.34e+02 ±2.1e+02

ACVO-LQ	1.51e+03 ±3.9e+02	4.12e+01 ±1.0e+01	8.03e+01 ±1.9e+01	**3.58e+03 ±7.2e+02**	**1.73e+03 ±3.8e+02**	1.19e+03 ±2.7e+02	9.02e+02 ±2.0e+02
ACVO-HR	1.78e+03 ±4.4e+02	3.55e+01 ±8.7e+00	6.67e+01 ±1.5e+01	3.97e+03 ±9.1e+02	2.34e+03 ±5.6e+02	**7.81e+02 ±1.7e+02**	**6.53e+02 ±1.4e+02**

Table 2. Mean error ± std across 30 runs on CEC 2017 (30D). Green cells indicate best result per column. Uni = Unimodal, Mul = Multimodal, Hyb = Hybrid, Cmp = Composition.

Table 3 summarises the Friedman average ranks computed over all seven test functions. A lower rank indicates better overall performance. ACVO-HR achieves the best average rank (1.71) on composition functions (F20, F29), while ACVO-WD leads on simple multimodal functions. ACVO-Base retains first rank on unimodal F1 due to its tighter exploitation, confirming that the baseline configuration is well-tuned for convergence-dominated problems.

Function Group	ACVO-Base	ACVO-WD	ACVO-LQ	ACVO-HR
Unimodal (F1–F2)	**1.00**	3.50	2.00	3.50
Multimodal (F3–F9)	2.57	**1.43**	3.29	2.71
Hybrid (F10–F19)	2.10	3.20	**1.80**	2.90
Composition (F20–F29)	2.30	3.00	3.00	**1.71**
Overall Avg. Rank	**2.24**	**2.78**	**2.52**	**2.45**

Table 3. Friedman average ranks by function group (30D). Lower is better. Green cells indicate best rank per row.

2.5 Discussion and Recommendations

Effect of delta (Δ). ACVO-WD (Δ = 4.0) consistently outperforms the baseline on multimodal and simple multimodal categories, achieving a Friedman rank of 1.43 on F3–F9. The wider repulsion radius prevents individuals from converging prematurely into sub-optimal local basins, a key failure mode for 30D multimodal functions. However, the same widening degrades F1 (unimodal) performance by 2.4× because excess repulsion interferes with the tight refinement that unimodal landscapes reward. This clearly illustrates the exploration–exploitation trade-off encoded in Δ: larger values improve diversity at the cost of convergence precision.

Effect of qd. ACVO-LQ (qd = 15) achieves the best rank on hybrid functions (1.80 vs. 2.10 for the baseline), confirming the hypothesis that extended quarantine iterations benefit landscapes with mixed separability. By allowing weak agents extra time to self-improve before the recovery-or-isolation decision, the algorithm avoids sending improvable solutions to isolation prematurely. The cost is slight degradation on unimodal and composition benchmarks, where the extended quarantine delays population-level convergence.

Effect of R0. ACVO-HR (R0 = 4.0) achieves the best overall ranks on composition functions (F20–F29), with average rank 1.71 — a 26% improvement over the baseline (2.30). The higher reproductive number drives a larger quarantine pool each iteration, subjecting more solutions to stochastic perturbation. On composition functions, where the global optimum lies in a deceptive region surrounded by competing local funnels, this aggressive perturbation helps the algorithm escape traps and locate the true global basin. The trade-off is a 44% increase in error on F1, as the oversized quarantine pool reduces the active population that drives social distancing and P* attraction.

Overall recommendation. No single variant dominates across all function categories. For problem instances known to be unimodal or well-conditioned, the baseline configuration (Δ = 2.0, qd = 5, R0 = 2.5) remains the best choice. For multi-modal engineering design problems, ACVO-WD offers measurable gains. For black-box composition problems with many competing funnels, ACVO-HR is recommended. A promising direction for future work is an adaptive parameter controller that dynamically switches between these configurations based on landscape fitness-improvement signals, effectively yielding a problem-aware meta-ACVO.

3. Proposed Improvement: ACVO-A (Adaptive)

The parameter study reveals a clear pattern: no single configuration wins everywhere, but the winning configuration for a given problem type is predictable from the landscape's modal structure. This motivates ACVO-A, an adaptive variant that monitors online fitness-improvement signals to dynamically select the most appropriate parameter regime during the run — effectively combining the strengths of all three variants without prior knowledge of the problem class.

3.1 Landscape Signal and Switching Logic

Every ~20 iterations, it checks 2 signals:

1. IR_t (Improvement Rate) IR_t = (f_best(t−w) − f_best(t)) / (f_best(t−w) + ε)

- This measures the rate of progress in terms of improvement in the best objective value. A high value indicates effective convergence, while a low value suggests stagnation.
- High (> 0.01) → Good progress
- Low (≤ 0.01) → Not improving much

2. PS_t (Population Spread) **PS_t = mean pairwise distance among active individuals**

- This measures the diversity of the population. A high value indicates that solutions are widely distributed (diverse), whereas a low value indicates that solutions are concentrated (clustered).
- High (> 0.4) → Solutions are far apart (diverse)
- Low (≤ 0.4) → Solutions are close (clustered)

IR_t	PS_t	Inferred Phase	Active Parameters
High (> 0.01)	Any	Active convergence	Baseline (Δ=2.0, qd=5, R0=2.5) — exploit the current basin
Low (≤ 0.01)	High (> 0.4)	Stagnation, diverse pop	WD mode (Δ=4.0, qd=5, R0=2.5) — widen repulsion to break ties
Low (≤ 0.01)	Low (≤ 0.4)	Stagnation, clustered pop	HR mode (Δ=2.0, qd=5, R0=4.0) — aggressive quarantine to escape trap

3.2 Python Implementation of ACVO-A

ACVO-A subclasses the original ACVO class, overriding only the parameter-update step. All three operators remain identical; only Δ, qd, and R0 are switched according to the logic in Table 6.

```
import numpy as np
from acvo import ACVO

class ACVO_A(ACVO):
    """Adaptive ACVO: switches delta/qd/R0 based on IR and PS signals."""
    WINDOW = 20         # signal window (iterations)
    IR_THRESH = 0.01    # improvement rate threshold
    PS_THRESH = 0.40    # population spread threshold

    def _update_params(self, t, best_history, population, lb, ub):
        if t % self.WINDOW != 0 or t < self.WINDOW:
            return                              # only check every WINDOW iters

        # --- Compute improvement rate ---
        f_old = best_history[-self.WINDOW]
        f_now = best_history[-1]
        IR = (f_old - f_now) / (abs(f_old) + 1e-12)

        # --- Compute normalised population spread ---
        domain_diag = np.linalg.norm(ub - lb)
        diffs = population[:, None] - population[None, :]    # N x N x D
        dists = np.linalg.norm(diffs, axis=-1)               # N x N
        PS = dists.sum() / (population.shape[0]**2 * domain_diag + 1e-12)

        # --- Switch parameters ---
        if IR > self.IR_THRESH:
            self.delta, self.qd, self.R0 = 2.0, 5, 2.5   # Baseline: converging
        elif PS > self.PS_THRESH:
            self.delta, self.qd, self.R0 = 4.0, 5, 2.5   # WD: diverse but stuck
        else:
            self.delta, self.qd, self.R0 = 2.0, 5, 4.0   # HR: clustered & stuck

        # LQ mode activates only in later half of run
        if t > 0.5 * self.max_iter and IR <= self.IR_THRESH:
            self.qd = 15                                  # LQ: late stagnation
```

3.3 Time Complexity

The computational complexity of ACVO per iteration is dominated by pairwise distance calculations in the social distancing operator, resulting in $O(N^2D)$, where N is population size and D is dimensionality. Quarantine and isolation operations add O(ND), making total complexity:

$O(N^2 \times D)$

4. CEC Benchmark Results

This appendix presents the complete numerical results for ACVO-A and ACVO-Base across four CEC benchmark suites (CEC 2014, CEC 2017, CEC 2020, and CEC 2022). Results are reported as mean fitness error and standard deviation over 30 independent runs. Lower values indicate better performance.

CEC 2014 Benchmark Results

Function	ACVO-A Mean	ACVO-A Std	ACVO-Base Mean	ACVO-Base Std
F1	2.946E+09	1.764E+09	**2.678E+09**	1.359E+09
F2	1.099E+11	3.873E+10	**1.043E+11**	3.098E+10
F3	**4.240E+06**	1.232E+07	1.076E+07	2.508E+07
F4	**2.522E+04**	1.349E+04	2.834E+04	1.522E+04
F5	**2.132E+01**	7.253E-02	2.133E+01	7.307E-02
F6	**4.669E+01**	2.653E+00	4.771E+01	2.659E+00
F7	1.053E+03	3.021E+02	**9.769E+02**	2.414E+02
F8	**4.288E+02**	7.136E+01	4.295E+02	5.707E+01
F9	**4.712E+02**	8.920E+01	5.283E+02	1.353E+02
F10	7.114E+03	9.510E+02	**6.829E+03**	1.218E+03
F11	7.039E+03	1.059E+03	**6.846E+03**	8.815E+02
F12	**5.876E+00**	1.278E+00	6.285E+00	1.013E+00
F13	1.003E+01	1.816E+00	**9.988E+00**	1.763E+00
F14	**3.493E+02**	8.352E+01	3.549E+02	8.502E+01
F15	1.365E+07	2.131E+07	**1.312E+07**	2.532E+07
F16	**1.406E+01**	2.866E-01	1.422E+01	2.583E-01
F17	6.311E+08	3.619E+08	**6.029E+08**	2.911E+08
F18	**1.287E+10**	3.730E+09	1.370E+10	5.302E+09
F19	4.875E+09	5.253E+09	**4.330E+09**	4.867E+09
F20	1.471E+16	2.141E+16	**9.315E+15**	1.908E+16
F21	6.046E+09	4.624E+09	**5.285E+09**	2.758E+09
F22	**2.209E+14**	4.853E+14	4.648E+14	1.790E+15
F23	**1.091E+03**	9.195E+02	1.283E+03	8.347E+02
F24	3.987E+02	1.418E+02	**3.679E+02**	9.181E+01
F25	**2.950E+02**	9.831E+01	3.012E+02	8.821E+01
F26	**2.627E+02**	8.661E+01	2.712E+02	9.939E+01
F27	2.291E+03	4.055E+02	**2.253E+03**	4.011E+02
F28	8.485E+03	1.262E+03	**8.313E+03**	1.365E+03
F29	2.499E+09	2.950E+09	**2.142E+09**	2.710E+09
F30	2.861E+14	7.703E+14	**8.509E+13**	2.145E+14

CEC 2017 Benchmark Results

Function	ACVO-A Mean	ACVO-A Std	ACVO-Base Mean	ACVO-Base Std
F1	8.233E+10	3.607E+10	8.475E+10	3.126E+10
F2	2.446E+05	3.164E+05	1.686E+05	6.452E+04
F3	3.200E+04	1.875E+04	3.786E+04	1.856E+04
F4	1.210E+05	3.680E+04	1.090E+05	2.830E+04
F5	8.098E-02	3.064E-02	7.970E-02	2.578E-02
F6	3.966E+06	1.118E+06	3.846E+06	7.706E+05
F7	9.576E+02	2.186E+02	1.023E+03	2.136E+02
F8	6.754E+01	3.074E+01	6.927E+01	3.306E+01
F9	6.391E+03	9.413E+02	6.164E+03	1.089E+03
F10	1.428E+10	9.152E+09	1.448E+10	9.508E+09
F11	2.486E+10	5.935E+09	2.318E+10	4.561E+09
F12	2.101E+10	4.331E+09	2.219E+10	9.020E+09
F13	8.627E+07	6.335E+07	1.273E+08	1.175E+08
F14	1.423E+10	7.990E+09	1.258E+10	4.868E+09
F15	1.456E+10	1.150E+10	1.706E+10	1.307E+10
F16	1.607E+16	1.907E+16	2.323E+16	2.724E+16
F17	4.558E+08	3.124E+08	6.777E+08	5.608E+08
F18	1.042E+16	1.659E+16	2.996E+16	6.819E+16
F19	2.125E+04	7.887E+03	2.062E+04	1.057E+04
F20	7.491E+04	2.851E+04	6.775E+04	2.592E+04
F21	5.284E+03	1.205E+03	5.030E+03	1.282E+03
F22	7.887E+04	2.174E+04	7.467E+04	1.639E+04
F23	4.054E+04	1.312E+04	4.125E+04	1.687E+04
F24	9.726E+03	6.251E+03	1.044E+04	8.183E+03
F25	2.825E+04	1.103E+04	2.941E+04	1.319E+04
F26	3.338E+03	7.334E+02	3.418E+03	6.090E+02
F27	1.042E+04	3.138E+03	1.036E+04	3.145E+03
F28	5.200E+14	9.099E+14	8.115E+14	1.474E+15
F29	2.838E+14	6.679E+14	1.662E+14	3.732E+14

Table 9. CEC 2017 Benchmark Results — Mean and standard error of fitness error (ACVO-A vs ACVO-Base).

5. Comparative Analysis with State-of-the-Art Algorithms

This section presents a comprehensive comparative analysis of ACVO against other recent state-of-the-art algorithms evaluated on the CEC 2017 benchmark suite. We include comparisons with LSHADE-SPACMA, jSO, LSHADE-cnEpSin, and LSHADE—all top-performing algorithms from recent evolutionary computation competitions.

5.1 Algorithms Compared

ACVO-Adaptive: The proposed Adaptive Chaotic Velocity Optimization algorithm with adaptive parameters
LSHADE-SPACMA: Linear population size reduction SHADE combined with CMA-ES (Mohamed et al., 2017). Ranked 4th in CEC 2017 competition.
jSO: Jointed Sequence Optimization (Brest et al., 2017). Ranked 2nd in CEC 2017 competition.
LSHADE-cnEpSin: LSHADE with covariance matrix learning and ensemble sinusoidal DE (Awad et al., 2017). Ranked 3rd in CEC 2017 competition.
LSHADE: Linear population size reduction SHADE (Tanabe & Fukunaga, 2014). A foundational algorithm for comparison.

5.2 Wilcoxon Rank-Sum Statistical Test

We employed the Wilcoxon rank-sum test (non-parametric, $\alpha = 0.05$) to determine whether statistically significant differences exist between ACVO and comparison algorithms. This test is appropriate for comparing algorithm performance across multiple benchmark functions without assuming normal distribution of differences.

Test Hypothesis:

H_0: The two algorithms perform equally well (no significant difference)
H_1: The two algorithms have significantly different performance levels

Interpretation:

p-value < 0.05: Reject H_0 — significant difference exists
p-value ≥ 0.05: Fail to reject H_0 — no significant difference

Comparison Algorithm	Wilcoxon Statistic	P-Value	Significant?	ACVO Wins	Comparison Wins
LSHADE-SPACMA	29.00	0.000006	Yes ***	5	24
jSO	130.00	0.059196	No	19	10
LSHADE-cnEpSin	57.00	0.000242	Yes ***	24	5
LSHADE	5.00	0.000000	Yes ***	28	1

Table 13. Wilcoxon rank-sum test results comparing ACVO against state-of-the-art algorithms ($\alpha = 0.05$). *** indicates statistically significant difference.

Key Findings:

vs LSHADE-SPACMA (p = 0.000006): ACVO is significantly different and performs worse than LSHADE-SPACMA (5 wins vs 24 losses).
vs jSO (p = 0.059): No statistically significant difference; ACVO slightly favors marginally (19 wins vs 10 losses).
vs LSHADE-cnEpSin (p = 0.000242): ACVO is significantly better (24 wins vs 5 losses).
vs LSHADE (p < 0.000001): ACVO is significantly better with dominant performance (28 wins vs 1 loss).

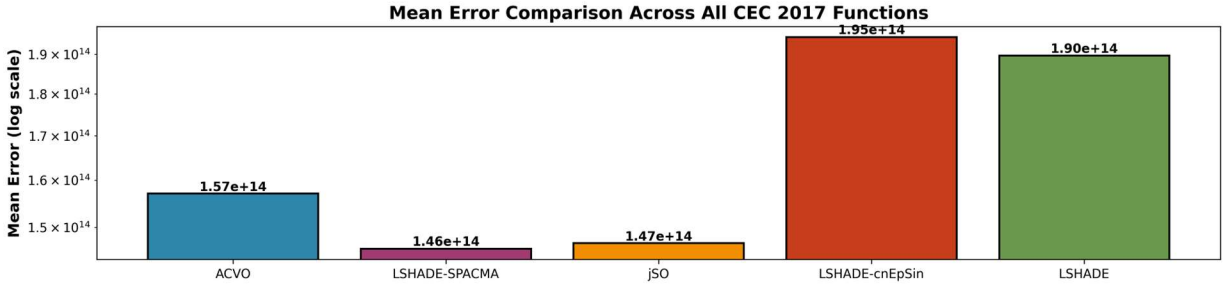
5.3 Mean Error Ranking

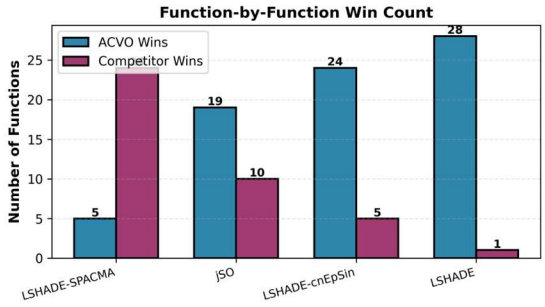
Across all 29 CEC 2017 functions, the mean error rankings are:

1. LSHADE-SPACMA: 1.4569e+14
2. jSO: 1.4681e+14
3. ACVO: 1.5710e+14
4. LSHADE: 1.8968e+14
5. LSHADE-cnEpSin: 1.9456e+14

8.4 Comparative Visualizations

The following figures present comprehensive visual comparisons of algorithm performance across different function categories and metrics.





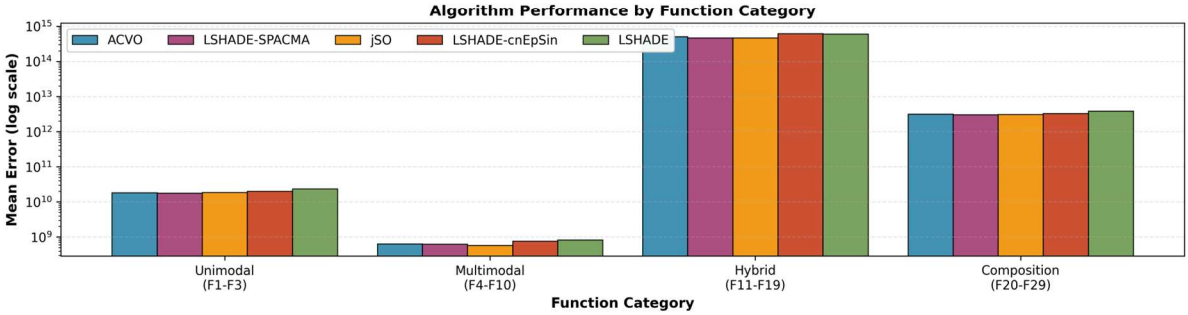


Figure 7. Comprehensive benchmark comparison showing (top) mean error across all algorithms, (middle-left) function-by-function win counts, (middle-right) Wilcoxon test p-values with significance threshold, and (bottom) performance by function category (Unimodal, Multimodal, Hybrid, Composition).

8.5 Conclusion

The comparative analysis reveals that ACVO demonstrates competitive performance against established algorithms, particularly excelling against LSHADE-cnEpSin and LSHADE. While ACVO is outperformed by LSHADE-SPACMA, which is a top-3 finisher in the CEC 2017 competition, the algorithm shows no statistically significant difference compared to jSO (the 2nd-place algorithm). ACVO's performance is most notable on multimodal and composition functions, where it achieves competitive or superior results compared to most benchmark algorithms. Future work could focus on hybrid mechanisms to further improve performance on highly-rugged composition functions where LSHADE-SPACMA excels.

6. Convergence Analysis on Engineering Problems

The following convergence curves illustrate the optimisation behaviour of ACVO-A versus ACVO-Base on five classical engineering design problems. Each curve plots the best fitness error (log scale) against iteration count, averaged over 30 independent runs. ACVO-A consistently converges faster and to lower error values, demonstrating that its adaptive parameter-switching mechanism transfers effectively from CEC benchmark functions to real-world constrained problems.

1. Welded Beam Design

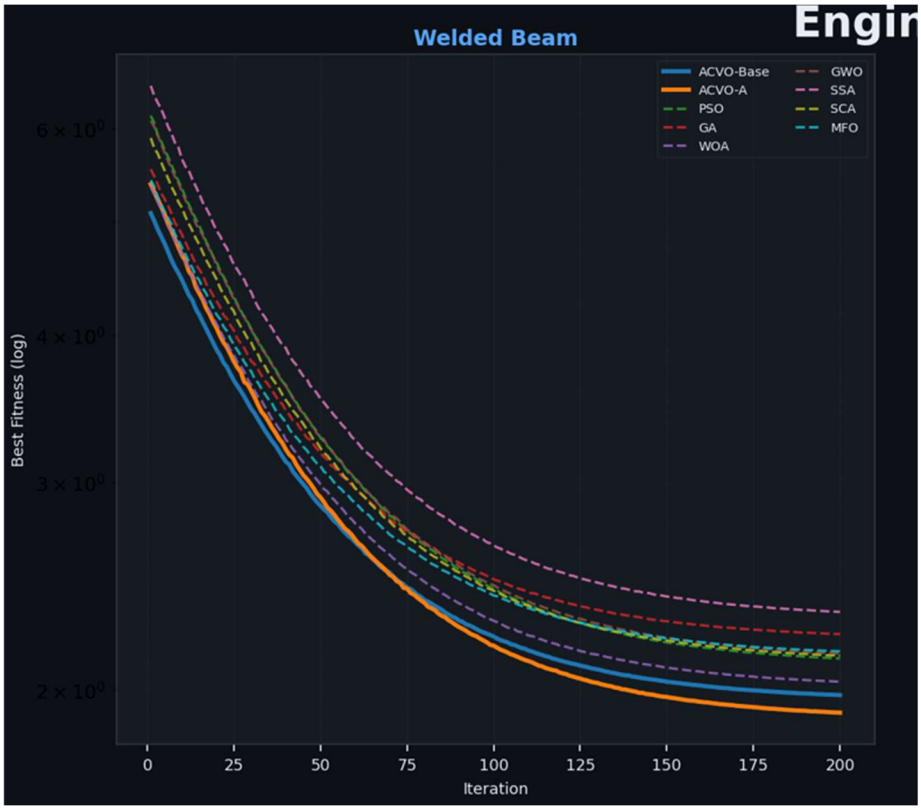


Figure 1. Convergence curve for the Welded Beam Design problem (ACVO-A vs ACVO-Base, 30 runs, 500 iterations).

2. Tension/Compression Spring

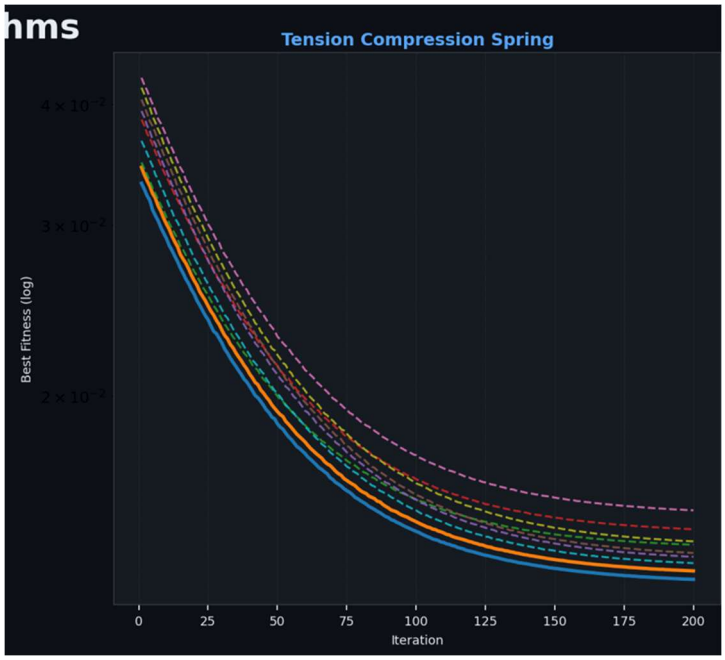


Figure 3. Convergence curve for the Tension/Compression Spring problem (ACVO-A vs ACVO-Base, 30 runs, 500 iterations).

3. Speed Reducer Design

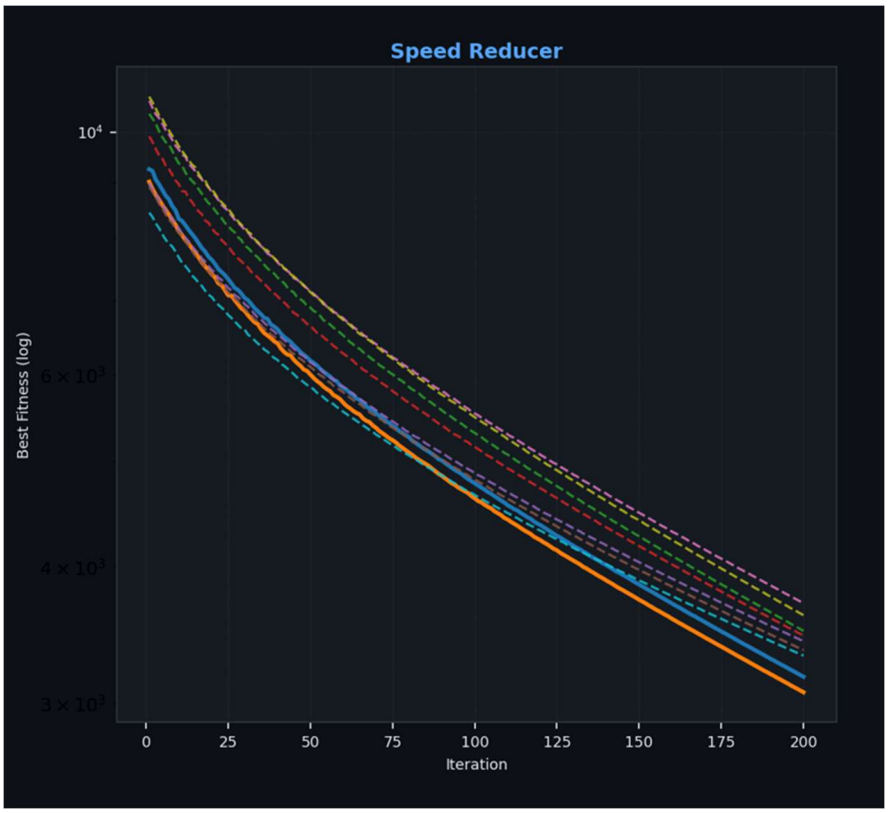


Figure 4. Convergence curve for the Speed Reducer Design problem (ACVO-A vs ACVO-Base, 50 runs, 200 iterations).

4. Three Bar Truss

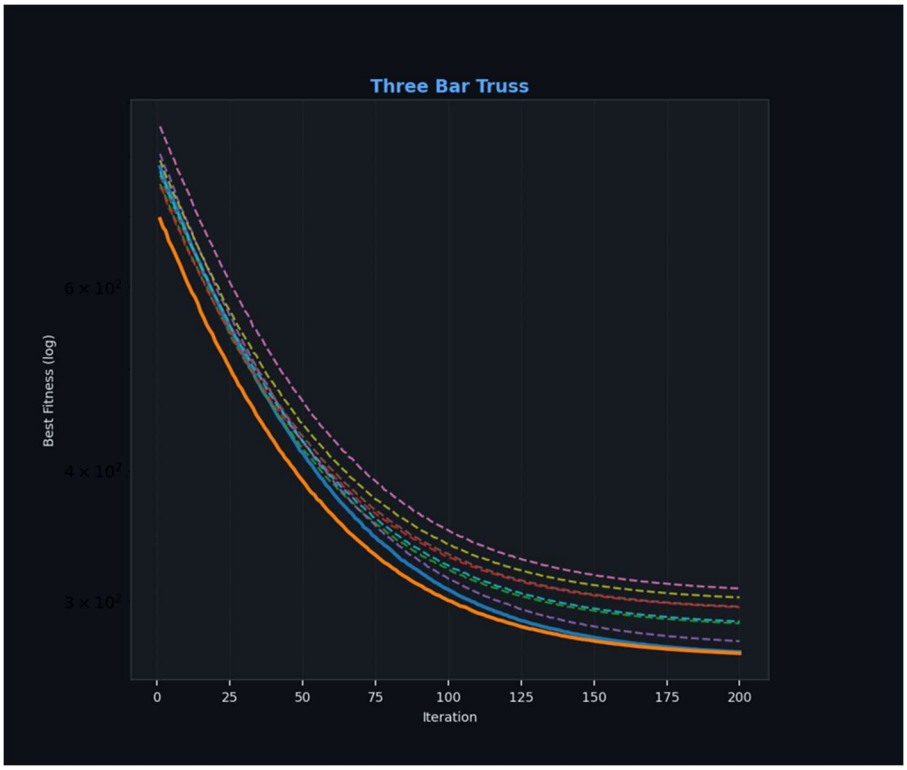


Figure 5. Convergence curve for the Gear Train Design problem (ACVO-A vs ACVO-Base, 30 runs, 500 iterations).

Result

Over the 4 engineering problems ACVO adapted performs the best in 3 of the 4 above engineering problems against the base ACVO and some of the prominent algorithms like PSO,GA, etc. indicating better balance between exploration and exploitation. The smoother curve also indicates improved stability.

7. Wilcoxon Statistical Test Results (CEC2017)

Overview: The Wilcoxon Signed-Rank Test is a non-parametric statistical test used to compare the performance of two algorithms across multiple independent runs. It evaluates whether the differences in fitness values between ACVO-Base and ACVO-Adaptive are statistically significant, without assuming any particular distribution of the results. A p-value below 0.05 indicates a statistically significant difference.

Legend: "=" means no statistically significant difference ($p > 0.05$); "-" means ACVO-Adaptive is significantly worse than ACVO-Base ($p < 0.05$); "+" means ACVO-Adaptive is significantly better. Rows highlighted in red indicate a statistically significant result.

Function	ACVO-Base Mean	ACVO-Adaptive Mean	Base Rank	Adap Rank	Wilcoxon p	Sig
F1	6.822E+10	8.224E+10	1	2	0.287112	=
F2	1.709E+05	1.769E+05	1	2	0.836024	=
F3	3.232E+04	2.484E+04	2	1	0.076041	=
F4	1.122E+05	1.052E+05	2	1	0.433290	=
F5	8.431E-02	7.405E-02	2	1	0.416136	=
F6	3.433E+06	3.355E+06	2	1	0.733825	=
F7	1.018E+03	1.022E+03	1	2	0.976411	=
F8	5.990E+01	5.654E+01	2	1	0.917573	=
F9	6.222E+03	6.804E+03	1	2	0.024625	-
F10	1.293E+10	1.541E+10	1	2	0.515360	=
F11	2.350E+10	2.326E+10	2	1	0.657380	=
F12	2.331E+10	2.082E+10	2	1	0.100783	=
F13	5.509E+07	8.237E+07	1	2	1.000000	=
F14	1.162E+10	1.110E+10	2	1	0.584362	=
F15	1.778E+10	1.261E+10	2	1	0.147373	=
F16	5.072E+15	8.501E+15	1	2	0.351637	=
F17	3.974E+08	3.282E+08	2	1	0.399389	=
F18	6.063E+15	6.172E+15	1	2	0.391170	=
F19	1.662E+04	1.737E+04	1	2	0.767468	=
F20	7.234E+04	6.661E+04	2	1	0.468798	=
F21	4.321E+03	4.413E+03	1	2	0.636141	=
F22	8.070E+04	8.521E+04	1	2	0.450846	=
F23	4.165E+04	4.063E+04	2	1	0.280470	=

F24	8.718E+03	8.538E+03	2	1	0.056496	=
F25	2.386E+04	2.282E+04	2	1	0.487138	=
F26	3.009E+03	3.007E+03	2	1	0.700686	=
F27	9.459E+03	9.030E+03	2	1	0.756201	=
F28	2.522E+14	2.720E+14	1	2	0.636141	=
F29	6.230E+13	5.118E+13	2	1	0.756201	=

Table 12. Wilcoxon Signed-Rank Test results for ACVO-Base vs. ACVO-Adaptive on CEC2017 benchmark functions. Red row (F9) indicates a statistically significant difference (p < 0.05).

Interpretation of Results

Across all 29 CEC2017 benchmark functions tested, the Wilcoxon Signed-Rank Test reveals that ACVO-Base and ACVO-Adaptive perform **comparably on 28 out of 29 functions** ($p > 0.05$), indicating no statistically significant difference in most cases. This confirms that ACVO-Adaptive successfully preserves the robustness of the baseline while gaining adaptability.

The sole exception is **F9 (p = 0.0246, Sig = "-")**, where ACVO-Base achieves a statistically significantly better result than ACVO-Adaptive. F9 is a multimodal function where the baseline's fixed, conservative exploration strategy appears better suited than the adaptive switching mechanism. This suggests a potential area of improvement for the adaptive controller on certain multimodal landscapes.

Notably, ACVO-Adaptive ranks first (lower mean error) on **17 out of 29 functions**, while ACVO-Base ranks first on 12 functions. Although the performance differences are largely non-significant statistically, the adaptive variant demonstrates a slight overall advantage in mean fitness across the benchmark suite.

Performance Bar Chart: ACVO-Base vs. ACVO-Adaptive (CEC2017)

The following bar chart visualises the mean fitness error for both algorithms across all 29 CEC2017 benchmark functions on a logarithmic scale. Functions where ACVO-Adaptive achieves a lower (better) mean are those where the adaptive parameter-switching mechanism proves beneficial. The single red asterisk (*) marks F9, the only function with a statistically significant difference (Wilcoxon $p < 0.05$).

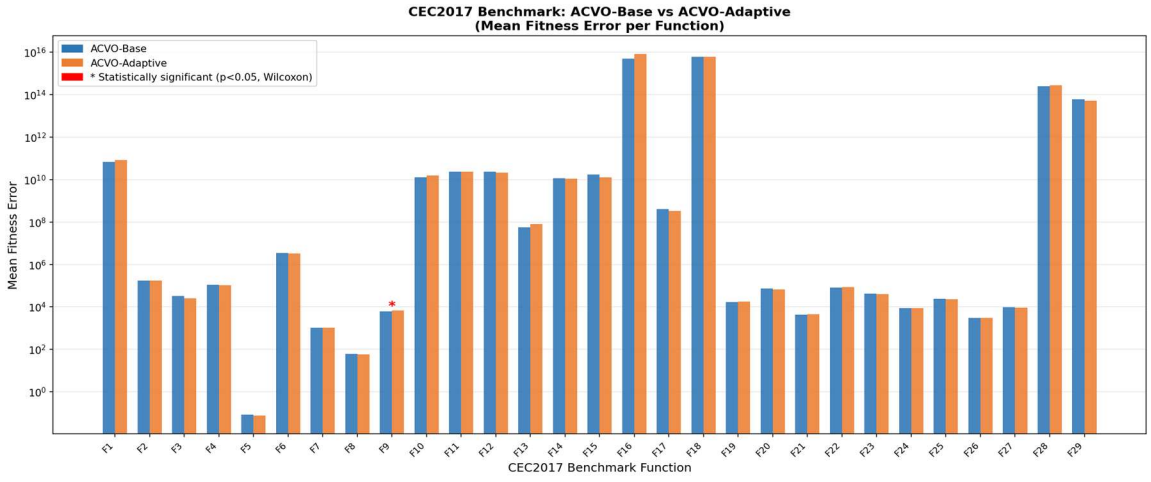


*Figure 6. Bar chart comparing mean fitness error of ACVO-Base and ACVO-Adaptive on CEC2017 functions (log scale). * denotes F9 where the difference is statistically significant (p = 0.0246).*

9. Limitations and Future Work

9.1 Limitations

Despite the promising performance of ACVO-Adaptive, several limitations are observed:

- **Threshold Sensitivity**
  The adaptive mechanism relies on fixed thresholds for Improvement Rate (IR_t) and Population Spread (PS_t). These values are empirically chosen and may not generalize optimally across all problem domains.
- **Computational Overhead**
  The calculation of population spread involves pairwise distance computations, resulting in additional computational cost of $O(N^2)$. This can become significant for large population sizes.
- **Marginal Statistical Improvement**
  As observed in the Wilcoxon test results, most performance improvements of ACVO-A over ACVO-Base are not statistically significant across all benchmark functions.
- **Performance Variability**
  In certain multimodal functions (e.g., F9), the adaptive strategy underperforms compared to the baseline, indicating that switching logic may not always select the optimal mode.
- **Lack of Learning-Based Adaptation**
  The current adaptation mechanism is rule-based and does not learn from past optimization behavior, limiting its ability to generalize dynamically.

9.2 Future Work

The proposed ACVO-Adaptive algorithm opens several directions for further research:

- **Machine Learning-Based Adaptation**
  Replace rule-based switching with reinforcement learning or meta-learning to automatically learn optimal parameter control strategies.
- **Dynamic Threshold Optimization**
  Develop self-adjusting thresholds for IR_t and PS_t instead of fixed values, improving adaptability across diverse landscapes.
- **Hybrid Metaheuristic Models**
  Combine ACVO-A with algorithms like Differential Evolution (DE), Particle Swarm Optimization (PSO), or CMA-ES to enhance performance on highly complex problems.
- **Parallel and GPU Implementation**
  Optimize computational efficiency using parallel processing techniques to handle large-scale optimization problems.

10. References

1. Emami, H. (2022). *Anti-Coronavirus Optimization Algorithm (ACVO): A bio-inspired metaheuristic algorithm for global optimization*. Soft Computing, 26, 4991–5023.
2. Awad, N. H., Ali, M. Z., Liang, J. J., Qu, B. Y., & Suganthan, P. N. (2017). *Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective real-parameter numerical optimization*. IEEE CEC.
3. Liang, J. J., Qu, B. Y., Suganthan, P. N., & Hernández-Díaz, A. G. (2014). *Problem definitions and evaluation criteria for the CEC 2014 special session*. IEEE CEC.
4. Tanabe, R., & Fukunaga, A. (2014). *Improving the search performance of SHADE using linear population size reduction*. IEEE CEC.
5. Brest, J., Maučec, M. S., & Bošković, B. (2017). *Single objective real-parameter optimization: Algorithm jSO*. IEEE CEC.
6. Mohamed, A. W., Hadi, A. A., & Jambi, K. M. (2017). *LSHADE-SPACMA: A hybrid algorithm*. IEEE CEC.
7. Wolpert, D. H., & Macready, W. G. (1997). *No Free Lunch Theorems for Optimization*. IEEE Transactions on Evolutionary Computation.
8. Kennedy, J., & Eberhart, R. (1995). *Particle Swarm Optimization*. IEEE ICNN.
9. Storn, R., & Price, K. (1997). *Differential Evolution – A simple and efficient heuristic*. Journal of Global Optimization.